

Jacqueline Schulz, Heike Fritz, Mareike Starker, Fabian Scholl, Fabian Meintzer

Moderne Softwareentwicklung –

Zwischendokumentation: Moderne Softwareentwicklung WS 2025/26 | Team 6 - Thema Digitale Bürgerabstimmungsplattform (eVote)

1. Projektziele und Kontext

1.1 Zielsetzung

Ziel des Projekts ist die Entwicklung einer digitalen Bürgerabstimmungsplattform (eVote), die als sicheres Online-Abstimmungssystem für Bürgerbefragungen dient. Bürger können über städtische Entscheidungen oder Projekte abstimmen.

Funktionale Anforderungen:

1. Nutzerregistrierung und Authentifizierung
2. Abstimmungsübersicht
3. Abstimmungsmechanismus
4. Ergebnisübersicht
5. Abstimmungssicherheit

1.2 Technologie-Stack

Für die Umsetzung wurde folgender Technologie-Stack gewählt:

- **Programmiersprache:** Python, Fast API (Flexibilität, umfangreiche Libraries)
- **Entwicklungsumgebung:** PyCharm (integrierte Git-Unterstützung, Debugging)
- **Testing Framework:** pytest (TDD-Unterstützung, parametrisierte Tests)
- **CI/CD:** GitHub Actions (nahtlose GitHub-Integration)
- **Code-Qualität:** SonarCloud (automatische Analyse, Technical Debt Tracking)
- **Validierung:** Pydantic (Type Safety, automatische Validierung)

2. Versionskontrolle mit Git (Übung 1)

Im Rahmen der ersten Übung wurde ein gemeinsames Git-Repository angelegt und ein Git-Handout für Anfänger erstellt. Die Arbeit erfolgte nach folgender Struktur:

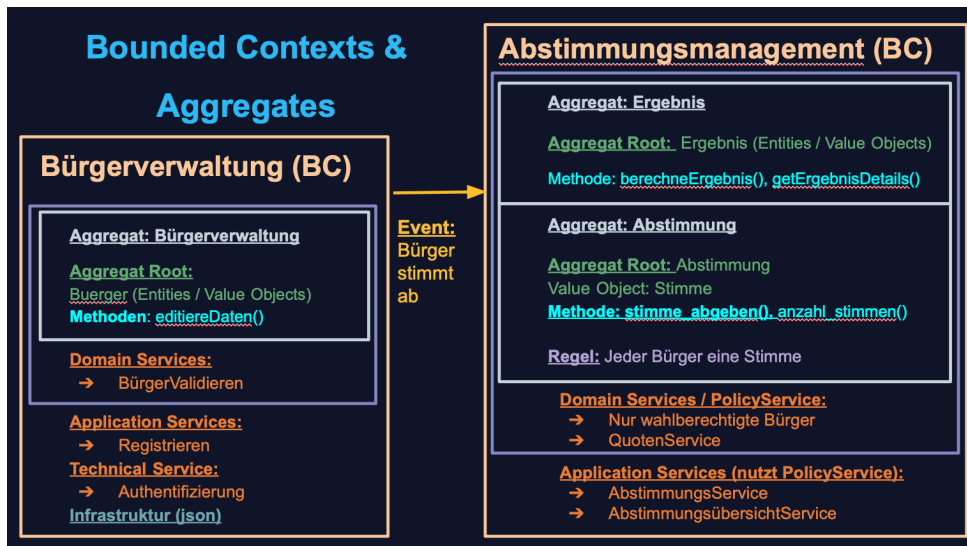
- **Repository-Setup:** Zentrales GitHub-Repository mit Zugriffsrechten für alle Teammitglieder
- **Branch-Strategie:** Jedes Teammitglied erstellte einen eigenen Feature-Branch
- **Collaborative Workflow:** Jeder Commit repräsentiert eine logisch abgeschlossene Änderung
- **Pull Requests (PR)** für jeden Beitrag
- **Code Reviews** durch andere Teammitglieder vor dem Merge

3. Domain-Driven Design (Übung 3)

3.1 Event Storming

Basierend auf Event Storming wurden vier zentrale Domain Events identifiziert. Bürger registriert, Abstimmung erstellt, Stimme abgegeben und Ergebnis berechnet.

3.2 DDD-Struktur



Es existieren zwei Bounded Contexts (BC): Im BC Bürgerverwaltung liegt die fachliche Domäne mit einem Domain Service „Bürger validieren“ und dem Aggregat „Bürger“, dessen Aggregate Root die Methode `editiereDaten()` bereitstellt. Die Domäne bildet hier den fachlichen Kern und enthält alle Regeln, die direkt die Konsistenz und Integrität des Bürgers betreffen. Außerhalb der Domäne befinden sich der Application Service „registrieren“ und der Technical Service „Authentifizieren“, die beide nicht zur fachlichen Logik gehören: Der Application Service orchestriert den Ablauf der Registrierung, während der Technical Service die technische Umsetzung der Authentifizierung übernimmt. Auch die Infrastruktur, dargestellt durch `db.json`, liegt außerhalb der Domain, da sie lediglich Persistenz sicherstellt und keinen fachlichen Inhalt modelliert.

Im BC Abstimmungsmanagement existieren zwei Aggregate: „Abstimmung“ und „Ergebnis“. Das Aggregat „Abstimmung“ enthält den Aggregate Root `Abstimmung` mit den Methoden `stimmeAbgeben()` und `anzahlStimmen()`, einschließlich der Regel „jeder nur eine Stimme“. Das Aggregat „Ergebnis“ stellt `berechneErgebnis()` und `getErgebnisDetails()` bereit. Die Domäne umfasst zudem die Domain Services „QuotenService“ und die Policy „Nur wahlberechtigte Bürger“, welche fachliche Regeln implementieren. Die Application Services „Abstimmungsservice“ und „AbstimmungsübersichtService“ liegen außerhalb der Domain, da sie lediglich Abläufe orchestrieren, z. B. das Sammeln von Stimmen und die Darstellung von Übersichten, ohne eigene fachliche Logik.

Die beiden BCs sind über das Event „Bürger stimmt ab“ gekoppelt, wodurch Aktionen in der Bürgerverwaltung fachlich relevante Folgen im Abstimmungsmanagement auslösen. Dadurch bleibt die fachliche Domäne innerhalb jedes BCs konsistent, während Application- und Technical Services die Abläufe und technischen Anforderungen orchestrieren.

4. Test-Driven Development und LLM-gestütztes Entwickeln (Übung 4)

4.1 TDD-Zyklus: Red-Green-Refactor

Im Projekt wird die Implementierung der Fachlogik durch Test-Driven Development (TDD) gesteuert. Dabei kommt ein kurzer, wiederholbarer Zyklus zum Einsatz, der aus den drei Schritten Red, Green und Refactor besteht.

Im eVote-Projekt wird dieser Zyklus zunächst bei der Ausgestaltung der Aggregate **Buerger**, **Abstimmung** und **Ergebnis** angewendet. Ein vollständiges Beispiel (Name-Validierung) illustriert den Ablauf konkret, während weitere Testarten – etwa für Datumsfeldern, Statusübergänge und Stimmenzählung – in komprimierter Form den breiten Einsatz des TDD-Ansatzes zeigen.

Beispiel: Name-Validierung

```
@pytest.mark.parametrize("name, valid", [
    ("Anna-Maria", True), # Happy Path
    ("Élodie", True), # Umlaute
    ("", False), # Negativ: leer
    ("Anna123", False), # Negativ: Zahlen
])
```

Implementierung der Validierungslogik in der Buerger-Klasse **buerger.py**:

```
@field_validator('name')
def validate_name(cls, v):
    pattern = r'^[a-zA-ZäöüÄÖÜß]+([ -][a-zA-ZäöüÄÖÜß]+)*$'
    if not re.match(pattern, v):
        raise ValueError('Ungültiger Name')
    return v
```

In den Modulen **abstimmung.py** und **ergebnis.py** wurden analog zur Bürgerverwaltung ebenfalls umfangreiche Tests implementiert, die z.B. Titel- und Beschreibungsvalidierung, konsistente Zeiträume, korrekte Statusübergänge, zulässige Stimmooptionen, nicht-negative und vollständige Stimmzahlen sowie die fehlerfreie Berechnung und Ausgabe der Abstimmungsergebnisse absichern.

4.2 LLM-Einsatz in der Entwicklung

In unserem aktuellen Projektstand setzen wir Large Language Models (LLMs) gezielt für Codegenerierung, Refactoring und Testunterstützung ein.

Am Beispiel der Domänenklasse "Ergebnis" hat das LLM zunächst beim Entwurf geholfen: Es generierte eine Pydantic-Klasse mit Feldern für Ergebnis-ID, Abstimmungs-ID, einer Liste von Stimmen und einem automatisch gesetzten Zeitstempel. Zusätzlich schlug es einen Validator für das Feld einzelwerte vor, der sicherstellt, dass die Liste der Stimmen nicht leer ist, andernfalls wird ein Fehler geworfen.

So ist fachlich garantiert, dass kein „leeres“ Abstimmungsergebnis entstehen kann:

```

class Ergebnis(BaseModel):
    ergebnisID: int
    abstimmungsID: int
    einzelwerte: List[Stimmenanzahl]
    timestamp: datetime = Field(default_factory=datetime.now)

@field_validator("einzelwerte")
@classmethod
def validate_einzelwerte(cls, einzelwerte):
    if not einzelwerte or len(einzelwerte) == 0:
        raise ValueError("Es müssen Stimmen für mindestens eine Option vorhanden sein.")
    return einzelwerte

```

In einem weiteren Beispiel fungierte das LLM als Pair-Programming-Partner und identifizierte im bestehenden Code des Authentifizierungsservices `authentifizierungs_service.py` eine Verbesserung; indem `datetime.utcnow()` durch `datetime.now(timezone.utc)` ersetzt werden sollte. Dadurch erzeugen wir jetzt „timezone-aware“ UTC-Zeitstempel für JWT-Tokens und vermeiden zukünftige Probleme mit Zeitzonen und Deprecation-Warnings (seit Python 3.12 als deprecated markiert):

```

❌ Vorher: Naive Zeitstempel
if expires_delta:
    expire = datetime.utcnow() + expires_delta
else:
    expire = datetime.utcnow() +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

✅ Nachher: Zeitzonen-sicher
now_utc = datetime.now(timezone.utc)
if expires_delta:
    expire = now_utc + expires_delta
else:
    expire = now_utc +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

```

4.3 Kritische Bewertung des LLM-Einsatzes

Vorteile: Mithilfe des LLM konnte in kurzer Zeit viel Code erstellt werden.

Dazu gehörten:

- Edge-Case-Identifikation (Tests)
- Code-Smell-Erkennung (Refactoring)
- Test-Generierung
- Best-Practice-Vorschläge

jedoch gibt es bei diesen vielen Vorteilen auch wesentliche **Nachteile/Grenzen**:

- LLM-Vorschläge sind nicht immer kontextgerecht und müssen kritisch geprüft werden
- Code muss vollständig verstanden werden - "Copy-Paste" ohne Verständnis führt zu Technical Debt
- Refactoring-Vorschläge können redundant oder unnötig komplex sein

Best Practice für die Arbeit mit KI: Wir sollten LLMs als Werkzeug nutzen, aber jede Zeile Code selber verstehen und verantworten können. Wir als Entwickler dahinter müssen bestehen bleiben: generierter Code sollte kritisch hinterfragt und getestet werden. So sollten nur überprüfte und Tests-bestätigte Refactorings im Code unseres Softwareprojekts landen.

5. CI/CD Pipeline (Übung 2)

5.1 GitHub Actions Workflow

Wir nutzen GitHub Actions für **Continuous Integration (CI)**. Bei jedem Push oder Pull Request auf den Branch **main** wird automatisch unsere Pipeline gestartet: Das Projekt wird zunächst gebaut und alle Python-Abhängigkeiten werden installiert. Anschließend führt **ruff** ein Linting durch, um Code-Stil, Lesbarkeit und einfache Fehler (z. B. unbenutzte Importe, tote Pfade) automatisch zu prüfen. Danach werden mit **pytest** alle Unit- und Integrationstests ausgeführt. Parallel messen wir mit **pytest-cov** die **Testabdeckung** und erzeugen einen Coverage-Report.

Im Anschluss werden die Ergebnisse an **SonarCloud** übergeben, wo Metriken wie Code Smells, Duplication, Complexity und Coverage ausgewertet werden. So sehen wir auf einen Blick, ob neue Commits die Codequalität verschlechtern oder Sicherheitswarnungen erzeugen.

Durch diese automatisierte CI-Kette stellen wir sicher, dass nur Änderungen in den main-Branch gelangen, die **baubar, getestet und qualitativ geprüft** sind. Fehler werden früh im Entwicklungsprozess sichtbar, Reviewer bekommen eine bessere Grundlage für Code-Reviews und die Qualität des eVote-Systems bleibt langfristig stabil und nachvollziehbar.

5.2 Herausforderungen und Lösungen

Aufgetretene Probleme:

- **YAML-Syntaxfehler:** Einrückungsfehler führten zu fehlerhaften Workflow-Ausführungen
- **Fehlende requirements.txt:** Pipeline konnte Abhängigkeiten nicht installieren
- **Exit Code 5:** Keine Tests gefunden, da Testdateien nicht korrekt benannt waren

Lösungsansätze:

- **test_sanity.py:** Hinzufügen eines minimalen Sanity-Tests zur Validierung der Pipeline-Konfiguration
- **Iterative Anpassung:** Schrittweise Verbesserung der YAML-Konfiguration basierend auf CI-Logs

6. SonarQube Cloud Analysis (Übung 5)

6.1 Integration in GitHub und die CI/CD-Pipeline

SonarQube Cloud wurde als zusätzlicher Schritt in die GitHub Actions Pipeline integriert:

- name: SonarCloud Scan
uses: SonarSource/sonarcloud-github-action@master

```
env:
  GITHUB_TOKEN:    ${ secrets.GITHUB_TOKEN }
  SONAR_TOKEN:    ${ secrets.SONAR_TOKEN }
```

Coverage-Reports: pytest generiert coverage.xml, die von SonarQube Cloud analysiert wird, um die Testabdeckung zu ermitteln

6.2 Analysierte Metriken

SonarQube Cloud liefert umfassende Code-Qualitätsmetriken (Beispiele):

- **Test Coverage:** Prozentuale Abdeckung des Codes durch Tests
- **Duplications:** Identifikation von dupliziertem Code
- **Security:** Bewertung der identifizierten Sicherheitslücken und Schwachstellen
- **Reliability:** Messung von Problemen, die die Zuverlässigkeit beeinträchtigen können, z.B. Bugs
- **Maintainability:** Messung von Problemen, z.B. Code Smells, die die Wartbarkeit beeinflussen können
- **Size:** Kennzahlen zur Größe des Projekts
- **Complexity:** Bewertet die Komplexität des Codes anhand von Kennzahlen wie zyklomatischer Komplexität

7. Bestandsaufnahme und Ausblick

7.1 Bisheriger Projektfortschritt

Im Projekt wurden ein konsistenter Git-Workflow mit Branch-Strategie und Pull-Request-Reviews etabliert, die Bounded Contexts nach DDD modelliert und eine event-basierte Kommunikation zwischen den Kontexten definiert. Zudem wurden zentrale Fachbereiche wie Bürgerverwaltung, Abstimmung und Ergebnisberechnung testgetrieben entwickelt, eine CI/CD-Pipeline mit automatisierten Tests und SonarCloud-Analyse aufgebaut sowie eine durchgängige Validierung der Aggregate Roots mit Pydantic umgesetzt.

7.2 Nächste Schritte

Der nächste Schritt ist die Entwicklung einer benutzerfreundlichen Weboberfläche, die auf dem Python-basierten Framework FastAPI basiert. Im Fokus steht dabei die Anbindung der Domain-Logik über klar definierte API-Endpunkte, sodass die Eingaben aus dem Frontend entgegengenommen und mittels Pydantic zuverlässig validiert werden. Bei fehlerhaften Eingaben sorgt die API dafür, dass strukturierte Fehlermeldungen erzeugt und gezielt an den jeweiligen Feldern des Frontends angezeigt werden, um dem Nutzer eine präzise und verständliche Rückmeldung zu ermöglichen.

7.4 Kritische Reflexion

Das Projekt war für das Team zunächst herausfordernd, da die in diesem Projekt eingesetzten Methoden und Werkzeuge nur in Ansätzen bekannt waren und eine intensive Einarbeitung erforderten. Durch die parallele Einführung von Pydantic, Pytest und GitHub

Actions mussten neue Technologien mit professionellen Entwicklungspraktiken kombiniert werden.

Trotz der steilen Lernkurve entstand eine produktive Teamdynamik: Strukturierte Übungen, enge Zusammenarbeit, gegenseitige Code Reviews und der gezielte Einsatz von LLMs als Lernhilfe haben den Wissensaufbau deutlich beschleunigt. Regelmäßige Abstimmungen, Pair-Programming und gemeinsames Troubleshooting bei Pipeline-Problemen halfen dabei, individuelle Wissenslücken auszugleichen.

Rückblickend hat sich die anfängliche Überforderung etwas gelegt, und es ist ein erstes Verständnis für das Zusammenspiel der Konzepte entstanden – auch wenn wir uns weiterhin aktiv bemühen, Schritt zu halten und die Themen in der Tiefe zu durchdringen.